

Kernel y Device Tree

Módulo 2 — Linux Embebido, Técnicas Digitales III

Ing. Marín Ariel

Auxiliares becarios: Fabrizio Carlassara (BIDOC), Pablo Gómez (BIDOC), Alexis Hernández de la Cruz (BIS)

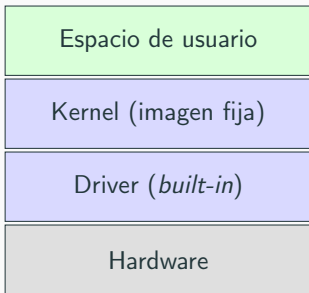
2026

UTN – Facultad Regional Avellaneda

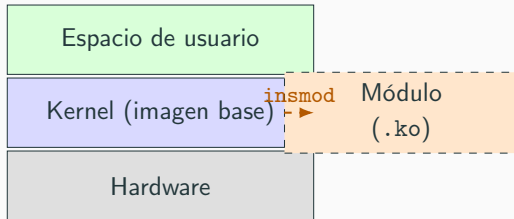
- Módulos cargables: qué son, qué pueden hacer y cómo se compilan
- Kconfig y Kbuild: de dónde vienen los CONFIG_*
- Device Tree: el problema que resuelve y su sintaxis básica
- Práctica: compilar y cargar un módulo en la Pi, leer el DT

El kernel: monolítico no es rígido

Driver integrado



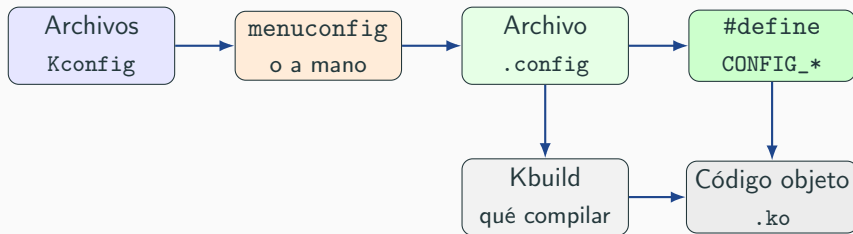
Módulo cargable



- Carga y descarga **sin reiniciar**: `insmod / rmod`
- Puede registrar drivers, crear archivos en `/dev`, acceder al hardware
- Desarrollo: cambiar el módulo lleva segundos; recompilar el kernel, minutos

Corre con el mismo privilegio que el kernel: un bug puede tirar todo el sistema.

Kconfig: de dónde vienen los CONFIG_*



Kbuild: compilar un módulo fuera del árbol

Qué declara nuestro Makefile:

- `obj-m += hello.o`
módulo a compilar
- `-C $(KDIR)`
cambiar al árbol del kernel
- `M=$(PWD)`
nuestro código está acá

```
make -C /lib/modules/$(uname -r)/build M=$(PWD) modules
```

Lo que hace Kbuild por nosotros:

- Busca cabeceras del kernel instalado
- Compila con los flags correctos
- Genera el `.ko` firmado

Demo en vivo

Práctica 1 — Módulo Hello World

```
insmod · rmmod · dmesg
```

Demo en vivo

Práctica 2 — Kconfig *out-of-tree*

Editar `.config` · recompilar · observar cambio en `dmesg`

Práctica 1 — Hello World

1. Instalar cabeceras del kernel
2. Escribir `hello.c` + `Makefile`
3. `make`
4. `sudo insmod hello.ko`
5. `dmesg | tail -5`
6. `sudo rmmod hello`

Práctica 2 — Kconfig

1. Agregar archivo `Kconfig`
2. Crear `.config` con
`CONFIG_HELLO_VERBOSE=y`
3. Actualizar `Makefile` con `-include .config`
4. Recompilar y recargar
5. Verificar en `dmesg`

USB anuncia quién es al conectarse.

Un UART soldado al SoC...

silencio total.

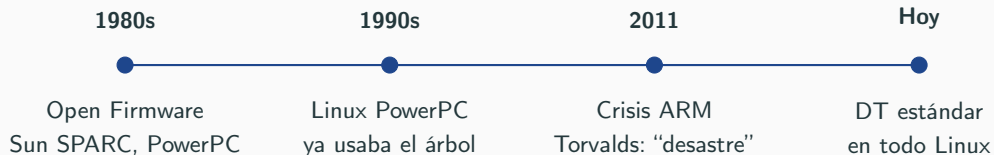
PCI / USB

- Bus con enumeración
- El dispositivo se presenta al encender
- Kernel carga el driver automáticamente

SoC (ARM, MIPS, RISC-V...)

- Periféricos mapeados en memoria fija
- Sin protocolo de enumeración
- ¿Quién le dice al kernel qué hay?

De Open Firmware al Device Tree



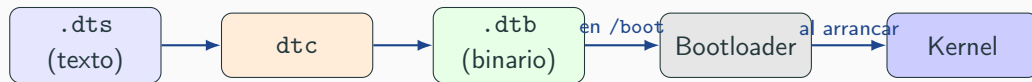
ARM no tenía convención de firmware $\Rightarrow$ miles de archivos `arch/arm/mach-*/board.c` casi idénticos.

Sintaxis DTS: los elementos clave

- **compatible:** lista de strings de más específico a más genérico; el kernel busca el driver por acá
- **reg:** dirección base y tamaño de los registros del periférico
- **status:** "okay" / "disabled"; controla si el periférico está activo
- **Etiqueta y phandle:** uart0: define una etiqueta; &uart0 la referencia desde otro nodo

```
uart0: serial@7e201000 {  
    compatible =  
        "arm,pl011";  
    reg = <0x7e201000  
        0x200>;  
    status = "okay";  
};
```

Pipeline del Device Tree



El kernel parsea el DTB y vincula
cada nodo con su driver via `compatible`

Demo en vivo

Práctica 3 — Inspeccionar el DTB de la Pi

`dtc -I dtb -O dts · explorar con less`

Demo en vivo

Práctica 4 — Leer el DT desde un módulo

```
of_find_compatible_node() · of_property_read_string()
```


Práctica 3 — Inspeccionar el DTB

1. `sudo apt install device-tree-compiler`
2. Descompilar el DTB activo con `dtc`
3. Buscar nodos UART: `compatible, status`
4. Explorar `/boot/firmware/overlays/`

Práctica 4 — Módulo lector

1. Módulo con `#include <linux/of.h>`
2. `of_find_node_by_path("/")` → modelo
3. `of_find_compatible_node("arm,pl011")` → UART
4. Misma ruta que vio `dtc` en la Pi

¿Preguntas?